OSPREYFLOW

Installation and Administration Guide

For whoever installs OspreyFlow and looks after it: **Lite** (one computer), **Small** (one cloud server on the cloud's managed services) and **Enterprise** (a Kubernetes cluster on the cloud's managed services). It describes what is built, and says plainly where something is not yet.

Version	1.7
Date	2026-10-19
Describes	OspreyFlow v0.4.0
Status	Issued

Document control

Version 1.7 · 2026-10-19 · For OspreyFlow v0.4.0 · Issued

Version	Date	What changed
1.7	2026-10-19	§2.4: the analysts' browsers need WebGL 2 (the graph renderer's version 4).
1.6	2026-10-18	§11: the HMP pack as a prison intelligence network, and adding it again after an update.
1.5	2026-10-15	A cover page, contents, a list of figures, and this version history.
1.4	2026-10-15	§16: a model download that stopped with `rename …-partial …`, and how to finish it.
1.3	2026-10-15	§11: the Enron email corpus as an optional download, with its safeguards.
1.2	2026-10-14	`ospreyflow repair-sign-in`; no step sends an operator to `make`.
1.1	2026-10-14	§14 rewritten: which installs `update` moves, and checking for and moving to a release.
1.0	2026-10-14	First issue in the install package, with screenshots.
0.4	2026-10-14	Enterprise built from your own computer: `install --mode enterprise --cloud`.
0.3	2026-10-14	Enterprise on Kubernetes: the cluster's namespace, storage classes and managed services.
0.2	2026-10-14	The guided install page ships with each release.
0.1	2026-10-13	Lite and Small, with every setting taken from the deployment's own schema.

Contents

How OspreyFlow is built 5

1. Choosing a mode 5

2. Before you start 6

- 2.1 Lite 6
- 2.2 Small 6
- 2.3 Enterprise 6
- 2.4 Checking a machine: ospreyflow doctor 6
- 2.4 The analysts' browsers 6

3. Getting the ospreyflow command 6

4. Installing Lite 7

- 4.1 The guided page: ospreyflow install --gui 7

5. Installing Small 7

- 5.1 What is made in your account 8
- 5.2 On the server, by hand 8

6. Installing Enterprise 8

- 6.1 What is made in your account 9

7. The answers file, install.yaml 9

8. Running it 10

- 8.1 Addresses 10
- 8.2 Where things are 10

9. People and accounts 11

- 9.1 The Admin area 11
- 9.2 Roles 11
- 9.3 Inviting someone 11

10. Models 12

11. Demo packs 12

12. Dashboards 13

13. Backup and restore 14

14. Updates 15

15. Scheduled and watched pipelines 16

16. When something is wrong 16

Appendix A — every osprey.yaml setting 17

List of figures

Figure 1 — Admin → People and roles: each person's roles, with Access, Disable and Reset sign-in 11
Figure 2 — A person's access opened under their row: clearance, unit and handling codes 12
Figure 3 — Admin → Demo packs: what each pack shows, where it was loaded, and the command for the rest 13
Figure 4 — Admin → Dashboards: registering a dashboard's account and seeing when it last pulled 14
Figure 5 — Superset at bi.: the boards and charts built on the reporting figures 14
Figure 6 — Admin → System: each component's health, the models in use, the version, and the last update check 15
Figure 7 — Admin → Schedules and runs 16

For whoever installs OspreyFlow and looks after it: **Lite** (one computer), **Small** (one cloud server on the cloud's managed services) and **Enterprise** (a Kubernetes cluster on the cloud's managed services). It describes what is built, and says plainly where something is not yet.

The other guides in this folder are the user guide (user-guide.pdf), for analysts, and the developer guide (developer-guide.pdf), for data engineers. Appendix A, every `osprey.yaml` setting, is made from the deployment's own schema, and is checked against it on every change.

How OspreyFlow is built

Three tiers; everything that passes between them crosses one of four typed ports:

Tier	What it is
The Glass	The browser workspace — the graph, map, timeline and reports, the Ingest Studio and the Admin area — one web application
The Conduit	The API (FastAPI). Every request is checked against the person making it: access control is applied inside each query to the stores, never after
The Vault	The stores: the graph store (ArcadeDB), the analytics database (DuckDB inside the Conduit, or ClickHouse shared by several Conduits), search (OpenSearch, or PostgreSQL full-text), PostgreSQL, the object store, and the models

Port	What passes through it
GraphPort	Typed traversal of the graph, with the person's clearance and handling injected into it
AnalyticsPort	Events, trends and histograms
ComputePort	Graph algorithms: centrality, communities, ownership chains
ModelPort	Extraction, vision, transcription and embedding, through the LiteLLM gateway

Two narrower ports sit beside them: the media store, for content files (an S3-protocol store — SeaweedFS or AWS S3 — or Azure Blob), and the recognition port.

One door in. A front proxy (Caddy) is the only published port: 80 on Lite, 80 and 443 with automatic HTTPS on Small and Enterprise. Behind it, at the deployment's address, are the Glass; sign-in (Keycloak) at `id.`; and dashboards (Superset) at `bi.`.

No route out by default. Every container sits on an internal network with no route to the internet. One joins the egress network only when a configured feature needs it: the model and speech-model downloads, the proxy's certificate, a client's own model provider, mail, an organisation's own sign-in, online map tiles, and on Small and Enterprise the cloud's managed services.

What the data is. Model-derived data stays *inferred* until a person confirms it. Records are never merged in storage: matching records are linked with a score and shown as one at query time. Every read is checked against the reader's clearance, unit and handling codes, and a hidden record cannot be detected from a count, a path length or a chart.

1. Choosing a mode

	Lite	Small	Enterprise
Where it runs	One computer (a laptop or a workstation)	One cloud server (AWS or Azure)	A Kubernetes cluster (EKS or AKS), three or more nodes
Who looks after the data	You, with `ospreyflow backup`	The cloud: managed PostgreSQL, S3 or Blob, daily backups	The cloud, for its managed services; the graph store in the cluster is not yet backed up (section 13)
Its address	`http://ospreyflow.localhost`	`https://<your domain>`	`https://<your domain>`
First sign-in	Demo accounts, passwords in `deployment.env`	A first administrator, by a one-time code	A first administrator, by a one-time code
Built with	`ospreyflow install --mode lite`	`ospreyflow install --mode small --cloud aws|azure`, from your own computer	`ospreyflow install --mode enterprise --cloud aws|azure`, from your own computer, or onto a cluster you already have

All three run the same product: the same containers, the same Glass, Ingest Studio and Admin area. Small swaps the deployment's own PostgreSQL and object store for the cloud's, and gets a real domain with HTTPS. Enterprise runs the same services as replicas across the cluster's nodes, the graph store and the analytics database as clusters of their own.

2. Before you start

2.1 Lite

- **Docker**: Docker Desktop on macOS and Windows (on Windows with the WSL 2 back end); Docker Engine with the Compose v2 plugin on Linux. Set it to start when you sign in: the stack comes back after a reboot only once Docker does.
- **Memory**: at least 16 GB available to Docker (32 GB is comfortable).
- **Disk**: at least 60 GB free, plus the demo packs you choose (PaySim is about 0.5 GB to download and more once loaded).
- **Git**, and **internet** for the first install.
- **Port 80 free** (the deployment's one door).

2.2 Small

On your own computer, from which you build the server:

- an **AWS or Azure account** you may create resources in;
- **OpenTofu** (`tofu`), which builds the server;
- the cloud's **command line** (`aws` or `az`), signed in to that account;
- **Git** and **uv**;
- a **DNS name** you control, such as `osprey.example.org`. If the name's zone is in the same cloud account (Route 53 or Azure DNS), the records are made for you.

The server itself needs nothing from you: it installs Docker and OspreyFlow on its first boot.

2.3 Enterprise

On your own computer:

- what Small needs (an account, OpenTofu, the cloud's command line signed in, Git, uv, a DNS name);
- **kubectl** and **Helm**;
- on Azure, **kubelogin** (`az aks install-cli` installs it): the cluster takes Entra ID sign-ins only;
- the **addresses you work from** (`api_cidrs`): the cluster's API, and on Azure the Key Vault, answer those and nothing else, so this computer must be inside one of them.

Onto a cluster you already have, only kubectl, Helm, Git and uv are needed, with kubectl pointed at it.

2.4 Checking a machine: `ospreyflow doctor`

`ospreyflow doctor --mode lite` checks, in the order to fix them: Docker installed, Docker running, Compose v2, memory, free disk, Git, uv and port 80. `ospreyflow doctor --mode small --cloud aws` (or `azure`) checks Git, uv, OpenTofu and the cloud's command line. `ospreyflow doctor --mode enterprise --cloud aws` adds kubectl and Helm (and on Azure kubelogin); without `--cloud` it checks kubectl, Helm and that a cluster answers. Each line is a tick, or a cross with what to do about it; the exit code is 1 when anything is missing. `install` runs the same checks first and stops, having changed nothing, if one fails.

2.4 The analysts' browsers

The graph and the map are drawn with **WebGL 2**, which every current Chrome, Edge, Firefox and Safari has (Safari from version 15, the others for longer). A browser from before about 2021, or one with hardware drawing turned off by policy, shows the rest of OspreyFlow but an empty canvas and map.

3. Getting the `ospreyflow` command

The command is installed once per computer by the bootstrap in the repository:

- **macOS and Linux**: `sh install.sh` from inside a checkout, or `OSPREYFLOW_REPO=<repository address> sh install.sh` to clone it into `~/ospreyflow`.
- **Windows**: `install.ps1` in PowerShell, the same way (`$HOME\ospreyflow`).

It installs **uv** if it is missing (from Astral's own installer), makes the checkout's Python environment, installs the command as a uv tool, and records where the checkout is (`~/.config/ospreyflow/home`, or `%APPDATA%\ospreyflow\home`). It starts nothing. If the command is not found afterwards, open a new terminal, or run `uv tool update-shell`.

The command finds its checkout by `OSPREYFLOW_HOME` first, then the recorded file, then by looking upwards from the folder you are in. Anything it prints that names a password, secret or token is hidden and replaced by a pointer to `deployment.env`.

4. Installing Lite

```
ospreyflow install --mode lite
```

asks a few questions — dashboards on or off, the folder the Ingest Studio lists, the folder for a basemap, which demo packs — then:

1. writes `osprey.yaml` from the answers (refusing, with nothing changed, if a different `osprey.yaml` is already there, unless `--force`);
2. on a release (the checkout sits on a `vX.Y.Z` tag): checks the release's signatures and runs exactly its images, pinned by digest; otherwise builds the images from the checkout;
3. starts the deployment and waits until every service is healthy;
4. makes the demo accounts (unless `--no-demo-accounts`) and loads the chosen demo packs.

It ends by saying the address and that the sign-in details are in `deployment.env`.

Without questions: `ospreyflow install --mode lite --yes` takes every default; `--config install.yaml` takes the answers from a file (section 7). **Building anyway**: `--build` builds from the checkout even on a release tag.

4.1 The guided page: `ospreyflow install --gui`

Opens a page in your browser, on this computer only, behind a one-time link (anyone else, or any other link, is refused). It walks through the same answers as screens:

- **Lite**: Mode, Folders, Profile, Components, Map (register a local basemap, build it, verify it), Models, Governance, Demo packs, Review (which runs the doctor), then Install, with the install's own output shown live.
- **Small and Enterprise**: Mode, Cloud and region, Domain, Components, Models, Governance, Demo packs, Review — which **saves** the answers: a Small server is built from a terminal (section 5), never from the page.

Everything is worked on a draft copy of `osprey.yaml`; the page closes itself when the install finishes, or after 30 idle minutes. On a release, the page is the release's own — fetched the first time, with its signature checked against the release; in a developer's checkout it is built with `cd apps/harness && npm ci && npm run build`, and the command says so if it is missing or out of date.

5. Installing Small

From your own computer:

```
ospreyflow install --mode small --cloud aws --config install.yaml
```

with at least:

```
mode: small
cloud:
  provider: aws            # or azure
  region: eu-west-2        # or uksouth, …
  domain: osprey.example.org
  dns_zone: example.org    # optional: make the DNS records in this Route 53 / Azure DNS zone
  admin_email: ops@example.org   # optional
  size: small              # small (32 GB server), medium (64 GB) or large (128 GB)
```

What happens:

1. **Plan.** OpenTofu shows everything it would make in your account (below). Nothing is made yet.
2. **Build, on your yes.** Answer `yes` (or pass `--yes`) and OpenTofu builds it. Its state is kept in `.cloud/small-<cloud>/` in your checkout, readable only by you. **Keep it, privately**: it is how the server is changed later, and on Azure it holds the database password.
3. **DNS.** With a `dns_zone`, the three records (`osprey.example.org`, `id.…` and `bi.…`) are made for you. Without one, the command prints the address to point them at and waits — up to an hour — until the name resolves to the server.
4. **The server installs itself.** On first boot it installs Docker and OspreyFlow, reads the database password (and on Azure the storage key) with **its own cloud identity**, installs onto the managed services, and gets its HTTPS certificate. The command

waits — up to 45 minutes — until `https://<domain>` answers. The server's own log is `/var/log/ospreyflow-first-boot.log`.
5. **The first administrator.** The server makes an account `admin` holding the Admin role and the highest clearance, with a **one-time code** instead of a password, and leaves the code in the cloud's secret store. The command reads it, **shows it once**, and deletes it from the store. At their first sign-in the administrator chooses their own password. Then they invite everyone else from the Admin area (section 9).

5.1 What is made in your account

	AWS	Azure
Server	One EC2 instance (r6i), encrypted disk, no SSH; reach it with Session Manager	One VM (E-series), no SSH port; Azure's service healing
Database	RDS for PostgreSQL 16, private, encrypted, its password kept by Secrets Manager	Azure Database for PostgreSQL, private, its password in Key Vault
Objects	S3: media, staging, site images, tile cache (private, encrypted)	Blob for media; the other stores on the server (SeaweedFS), covered by the server's backup
Secrets	Secrets Manager	Key Vault (purge protection on)
Recovery	EC2 automatic recovery; AWS Backup daily	Azure Backup daily
Open to the internet	80 and 443 only	80 and 443 only

Backups are kept 14 days by default (`backup_days`, 7–35).

5.2 On the server, by hand

The server's first boot runs, in effect, `ospreyflow install --mode small --config /etc/ospreyflow/install.yaml --yes --force --first-admin-code-file <file>` with the passwords in its environment. Run without `--first-admin-code-file`, it prints the first administrator's code once instead. `install --mode small` without `--cloud` and without the managed services in its answers is refused, saying which of the two you meant.

6. Installing Enterprise

From your own computer:

```
ospreyflow install --mode enterprise --cloud aws --config install.yaml
```

with at least:

```
mode: enterprise
cloud:
  provider: aws            # or azure
  region: eu-west-2        # or uksouth, …
  domain: osprey.example.org
  api_cidrs: [203.0.113.0/24]    # the addresses you work from; never 0.0.0.0/0
  admin_email: ops@example.org   # optional
  size: small              # small (3 nodes of 32 GB), medium (4 of 64 GB) or large (6 of 128 GB)
cluster:
  namespace: ospreyflow    # optional
```

What happens:

1. **Plan, then build on your yes**, as for Small (section 5): OpenTofu shows the cluster and the services it would make, and makes them when you answer `yes` (or pass `--yes`). Its state is kept in `.cloud/enterprise-<cloud>/`, readable only by you. **Keep it, privately**: it is how the cluster is changed later, and it holds secrets.
2. **What OpenTofu made becomes the answers.** It names the cluster's two storage classes and every managed service; these join your answers in `osprey.yaml`. Do not give `external:` or the storage classes yourself: with `--cloud` they are refused, since OpenTofu makes them.
3. **A kubeconfig of the install's own**, in the state folder; your `~/.kube/config` is not changed. Use it yourself with `KUBECONFIG=.cloud/enterprise-<cloud>/kubeconfig kubectl get pods -n ospreyflow`.

4. **The passwords and keys** of the managed services are read from the cloud's secret store with your own identity and handed to the install in its environment only — never on a command line, in `osprey.yaml` or on the screen.
5. **The install** puts the chart onto the cluster, makes its Secret (keeping every value already there when run again), configures sign-in, makes the first administrator, prints their one-time code once, and says the address to point `osprey.example.org`, `id.…` and `bi.…` at.

Run again, it changes nothing OpenTofu need not change, no password and no account.

Onto a cluster you already have: leave out `--cloud`, point kubectl at the cluster, and give the managed services yourself under `external:` and the cluster's classes under `cluster:` (section 7), with their passwords in the environment: `OSPREY_POSTGRES_PASSWORD`, `VALKEY_PASSWORD`, and for media on Blob `AZURE_STORAGE_KEY`. A site's own Helm values go in with `--chart-values <file>` (repeatable), applied over the rendered ones.

6.1 What is made in your account

	AWS	Azure
Cluster	EKS, its Kubernetes secrets encrypted with their own key; nodes in three private subnets	AKS, no local accounts (Entra ID only); a system pool and the deployment's own pool, three zones
Its API	Answers `api_cidrs` only	Answers `api_cidrs` only
Network policy	The VPC CNI's engine, so the chart's policies are enforced	Azure network policy
Database	RDS for PostgreSQL 16, two zones, password in Secrets Manager, deletion protection	PostgreSQL flexible server 16, zone-redundant, a delete lock, password in Key Vault
Cache	ElastiCache for Valkey, TLS and a token	Azure Cache for Redis, TLS only
Objects	S3 (as Small)	Blob for media; the other stores in the cluster
Search	Amazon OpenSearch Service (the install always asks for it; the module's own `search = false` leaves it to the cluster)	In the cluster
Shared volume	EFS (`ospreyflow-shared`)	Azure Files (`ospreyflow-shared`)
Store volumes	Encrypted gp3 (`ospreyflow-volumes`)	Zone-redundant premium disks (`ospreyflow-volumes`)

Every managed store is reached only from inside the cluster's network. Neither module has yet been built in a real account; the chart and the install have been proved in a local cluster.

7. The answers file, `install.yaml`

Every question `install` asks can be answered in this file; unknown settings are refused by name.

Setting	Meaning
`mode`	`lite`, `small` or `enterprise`
`components.dashboards`	Superset and report sync on (`true`, the default) or off
`folders.ingest`	The folder the Ingest Studio lists (absolute, or from `~`); default `~/OspreyFlow/ingest`
`folders.tiles`	Where a built basemap lives; default `~/OspreyFlow/basemap`
`models.local_only`	`true` (default): no case content leaves the deployment
`demo_packs`	A list of pack ids (section 11)
`paysim_file`	The downloaded PaySim file, when `paysim` is chosen
`mail.host`, `mail.port`, `mail.from`, `mail.starttls`, `mail.username`	A mail server for invitations (section 9.3); its password is never here
`cloud.provider`, `cloud.region`, `cloud.domain`, `cloud.size`, `cloud.dns_zone`, `cloud.admin_email`	Small (section 5) and Enterprise (section 6)
`cloud.api_cidrs`	Enterprise with `--cloud`: the addresses the cluster's API answers (section 6)
`cluster.namespace`, `cluster.storage_class`, `cluster.shared_storage_class`	Enterprise: the namespace, and onto a cluster you already have its storage classes (section 6)

Setting	Meaning
`external`	The managed services the deployment runs on — written by OpenTofu, or by you for a cluster you already have
`config`	Use this `osprey.yaml` exactly as it is, instead of `components`, `folders` and `models`

The answers become `osprey.yaml` (appendix A): `mode` sets `profile`, `components.dashboards` sets both `reporting.sync` and `reporting.superset`, and the address is `http://ospreyflow.localhost` for Lite and `https://<domain>` for Small and Enterprise.

8. Running it

Command	What it does
`ospreyflow start`	Brings the deployment up and waits for every service; `--build` rebuilds from the checkout first
`ospreyflow stop`	Stops every container; nothing is removed
`ospreyflow status`	Each service's state and health; exits 1 unless everything is running and healthy
`ospreyflow open`	Opens the Glass in your browser
`ospreyflow logs [service] [--follow] [--tail N]`	The services' logs (the last 200 lines by default)
`ospreyflow repair-sign-in [--no-demo-accounts]`	Registers the sign-in again — the identity provider's clients, roles and the demo accounts (leave them out with `--no-demo-accounts`); safe to run again
`ospreyflow uninstall [--keep-data] [--yes]`	Removes the containers; with `--keep-data` the data volumes stay, without it cases, accounts and indexes go too. Your checkout, `osprey.yaml` and your own folders are never touched

Every service restarts itself if it stops; nothing is registered with the operating system.

Enterprise is looked after with Kubernetes' own tools, not these verbs (which drive a Compose deployment): `kubectl get pods -n ospreyflow`, `kubectl logs`, and `helm uninstall ospreyflow -n ospreyflow` to remove it (the stores' volume claims are left in place, and the classes OpenTofu makes keep a volume even when its claim is deleted).

8.1 Addresses

One front proxy is the deployment's only door. Under the address in `network.public_origin`:

- the **Glass** — with the Ingest Studio and the Admin area inside it — at the name itself;
- **sign-in** (Keycloak) at `id.`;
- **dashboards** (Superset) at `bi.`, when dashboards are on.

Lite publishes port 80 and, on this computer only, `127.0.0.1:2480` for the graph store's own tools. Small publishes 80 (for the certificate) and 443. The Conduit's own port is published only with `network.expose_conduit: true`.

8.2 Where things are

Where	What
`osprey.yaml` (the checkout)	The deployment's settings; never a secret
`deploy/generated/<profile>/`	The deployment: `compose.yaml`, `deployment.env` (every password and key it made), `Caddyfile`, `glass.json`, `realm.json`, `litellm.yaml`, `versions.lock`, `RUNBOOK.md`, `state/`
`backups/`	Backup sets
`.install/`	The guided page's draft and answers; a release install's verified lock
`.cloud/`	OpenTofu's state for a Small server or an Enterprise cluster, and the cluster's kubeconfig — keep it private
Your ingest and basemap folders	As the answers named them

`RUNBOOK.md`, written for each deployment, lists what runs, its paths and its addresses.

9. People and accounts

Everyone signs in through the deployment's identity provider; the password is only ever typed on its own sign-in page. What a person may do is their **roles**; what they may see is their **clearance**, unit and handling codes, and the cases they are given.

9.1 The Admin area

Signed in with an administration right, choose **Admin** on Start. Each tab shows only to someone holding its right:

Tab	Right	What it does
People and roles	`manage_people`	Invite people; set their roles, clearance, unit and handling codes; disable, re-enable; reset a sign-in
Dashboards	`manage_dashboards`	Register a dashboard's account (its secret shown once), see when it last pulled, remove it
Schedules and runs	`operate_schedules`	Every scheduled or watched pipeline: pause, resume, run now, change its time, its runs
System	`administer_system`	Each component's health, the models in use, the version and whether an update exists
Demo packs	`administer_system`	Which packs are loaded, and the command that adds one

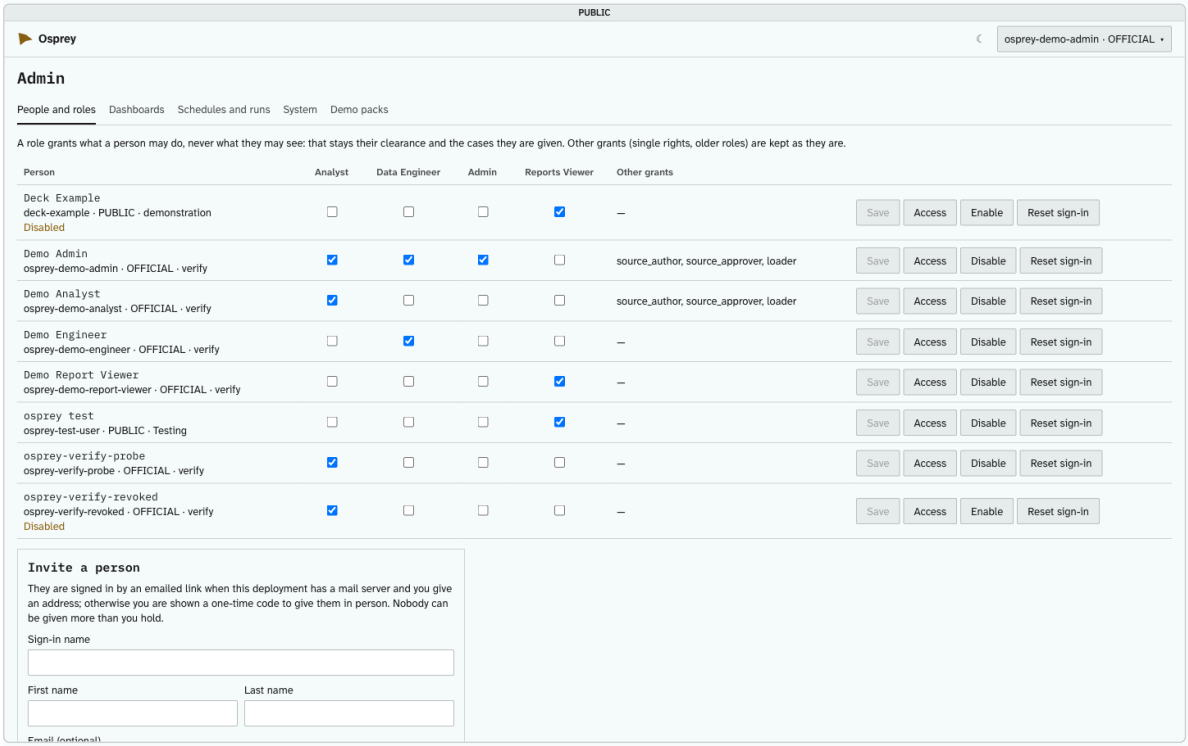


Figure 1. Admin → People and roles: each person's roles, with Access, Disable and Reset sign-in

9.2 Roles

Role	For
Analyst	Cases: investigate, edit the graph, confirm what models proposed, alerts and rules, exports, watches
Data Engineer	Pipelines: build, verify, publish, run and schedule them; the source files they load; held rows; screening lists. (It also carries the right to revert a load, which no screen or command uses yet)
Admin (`osprey-admin`)	The Admin area; the ontology and the map; the audit and export ledgers
Reports Viewer	Reports and data feeds; the alert queue, read-only

A single right can be given on its own. **Nobody can be given more than the person giving it holds** — clearance included — which is why the first administrator holds the highest clearance.

9.3 Inviting someone

Invite asks for a sign-in name, their name, optionally an email, a clearance, a unit, handling codes and roles. With a mail server configured and an email given, they get an email with a link to set their password. Otherwise you are shown a **one-time code**

(sixteen letters in four groups) to give them in person; it is shown once, and at first sign-in they choose their own password. **Reset sign-in** works the same way.

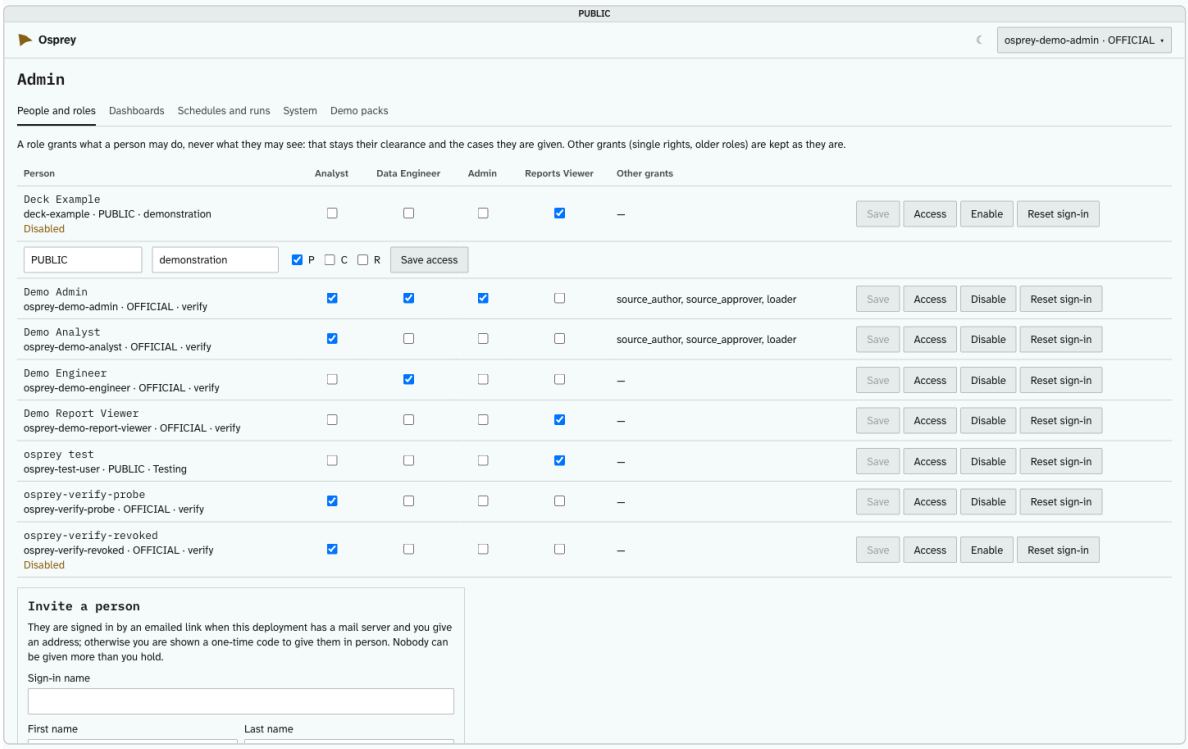


Figure 2. A person's access opened under their row: clearance, unit and handling codes

A mail server is set in `osprey.yaml`'s `mail:` (appendix A); if it signs in, its password is given once at install as `OSPREY_SMTP_PASSWORD` and kept only in `deployment.env`. With mail, the sign-in page also offers *Reset your password*.

10. Models

By default every model runs on the deployment itself (extraction, vision, transcription and embedding — appendix A, `models.*`) and **no case content leaves it**. A client with its own agreement with a model provider (OpenAI, Anthropic, Gemini, Azure, Bedrock…) may name it:

```
models:
  local_only: false
  sendable_handling_codes: [P]
  provider:
    name: openai
    roles: {extraction: gpt-4o}
```

The provider then **replaces** the local model for the roles named — for material whose handling code is in `sendable_handling_codes` only; anything else runs on the local model. Its key is given once at install as `OSPREY_MODEL_PROVIDER_KEY` and kept only in `deployment.env`. Admin → System shows each role's model, where it runs, and a notice that case content then leaves the deployment.

11. Demo packs

Id	What it shows	How it arrives
`hmp`	Prison intelligence: a prison's people and places, cell shares, co-defendants, conflicts, incidents, visits in person and by video, phones, calls and payments (synthetic)	Generated
`fixture-corp`	Network shapes, and duplicate records matched without being merged (synthetic)	Generated
`operation-ledger`	Preventive AML: a structuring ring, onboarding screened before it commits, ownership through a trust (synthetic)	Generated

Id	What it shows	How it arrives
`paysim`	Mobile-money fraud: 6.36 million simulated transactions with fraud labels	Downloaded by you
`enron`	**Real data**: about 150 people's real email at Enron, 1998–2002 — who wrote to whom, when, and about what	Downloaded by you; accepted before it loads; never by an install

Choose them at install (`demo_packs`), or later: `ospreyflow demo list`, `ospreyflow demo add <id>`. Each loads into its own case, with the installing administrator as Author. Adding **HMP** again is safe: a deployment that loaded it before v0.2.0 gains only its prison network, in the same case, and one that already has the network is left as it is. **PaySim** is a download: get `PS_20174392719_1491204439457_log.csv` from its Kaggle page and pass it with `ospreyflow demo add paysim --file <path>`; it is checked by size and checksum before anything starts. PaySim is CC BY-SA 4.0 (E. A. Lopez-Rojas, A. Elmir and S. Axelsson, 2016); its licence and attribution travel with it.

Enron is the one pack of **real personal data**: the email of real people, released during the US regulator's investigation and distributed by Carnegie Mellon University. OspreyFlow never carries it, and an install never adds it. To use it, download `enron_mail_20150507.tar.gz` (423 MB) from https://www.cs.cmu.edu/~enron/ yourself, then:

```
ospreyflow demo add enron --file enron_mail_20150507.tar.gz
```

It checks the file is exactly the May 7, 2015 version (size and SHA-256), then says what it is — real people's names, addresses and private correspondence; **no licence is stated**, CMU distributing it "as a resource for researchers" and asking that you "be sensitive to the privacy of the people involved"; your organisation needs its **own lawful basis** to process it — and loads it only when you type `yes`. (`--accept-real-data` accepts in a scripted run, and says so.) It is unpacked into the demo data folder — 150 mailboxes, about 517,000 messages and 2.5 GB, so allow 3 GB free with the download — with a record of the acceptance beside it, and loads into its own case, "Enron corpus (real data)", shared with no demo account but the administrator and analyst. The terms are in `packages/demo_fixtures/licences/enron/NOTICE.md`.

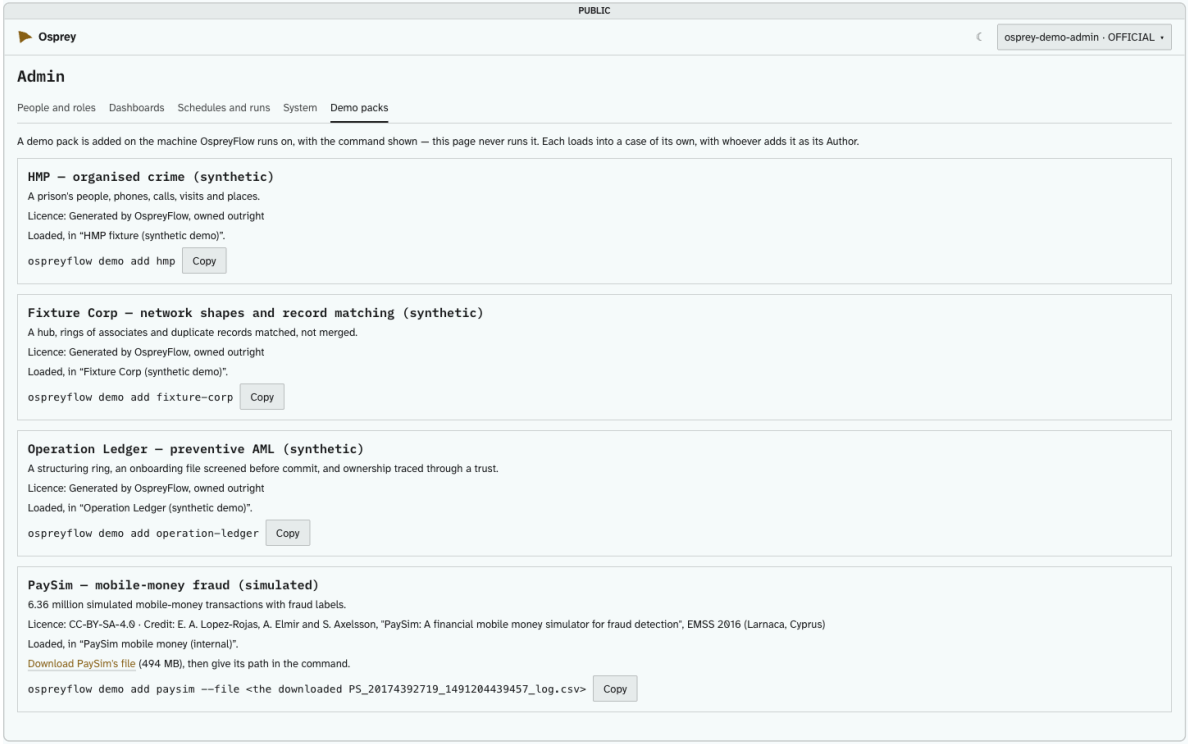


Figure 3. Admin → Demo packs: what each pack shows, where it was loaded, and the command for the rest

12. Dashboards

With dashboards on, **report sync** pulls each case's figures into a reporting database every 15 minutes (`reporting.sync_interval_seconds`), and **Superset** shows them at `bi.`, signing people in through the deployment's identity provider (below). A business intelligence tool (Power BI, Tableau, Qlik) reads the same figures:

1. register the dashboard in Admin → Dashboards and copy its secret (shown once);
2. add it to `REPORT_SYNC_DASHBOARDS` in `deployment.env` (a JSON list of `{"client_id", "secret"}`) and restart: `ospreyflow stop`, `ospreyflow start`;

3. a case's Author gives the dashboard's account Viewer on the case.

Each pull is made as that account, at its clearance, audited and recorded in the export ledger.

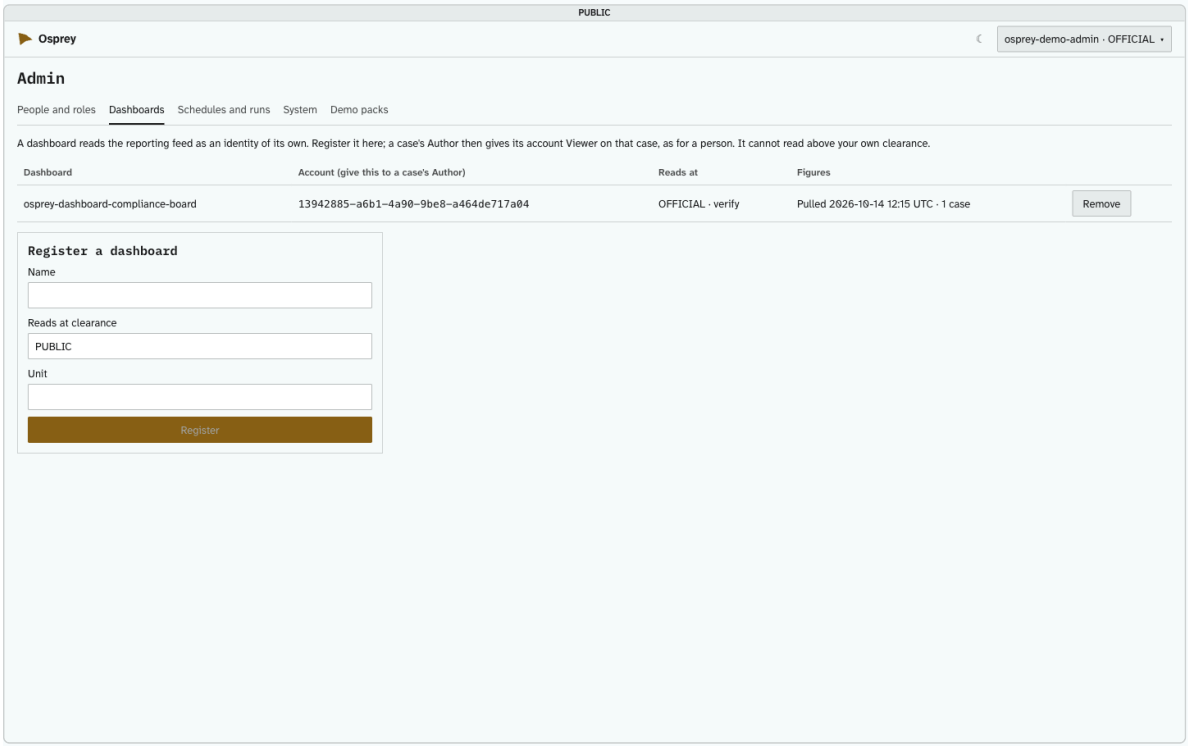


Figure 4. Admin → Dashboards: registering a dashboard's account and seeing when it last pulled

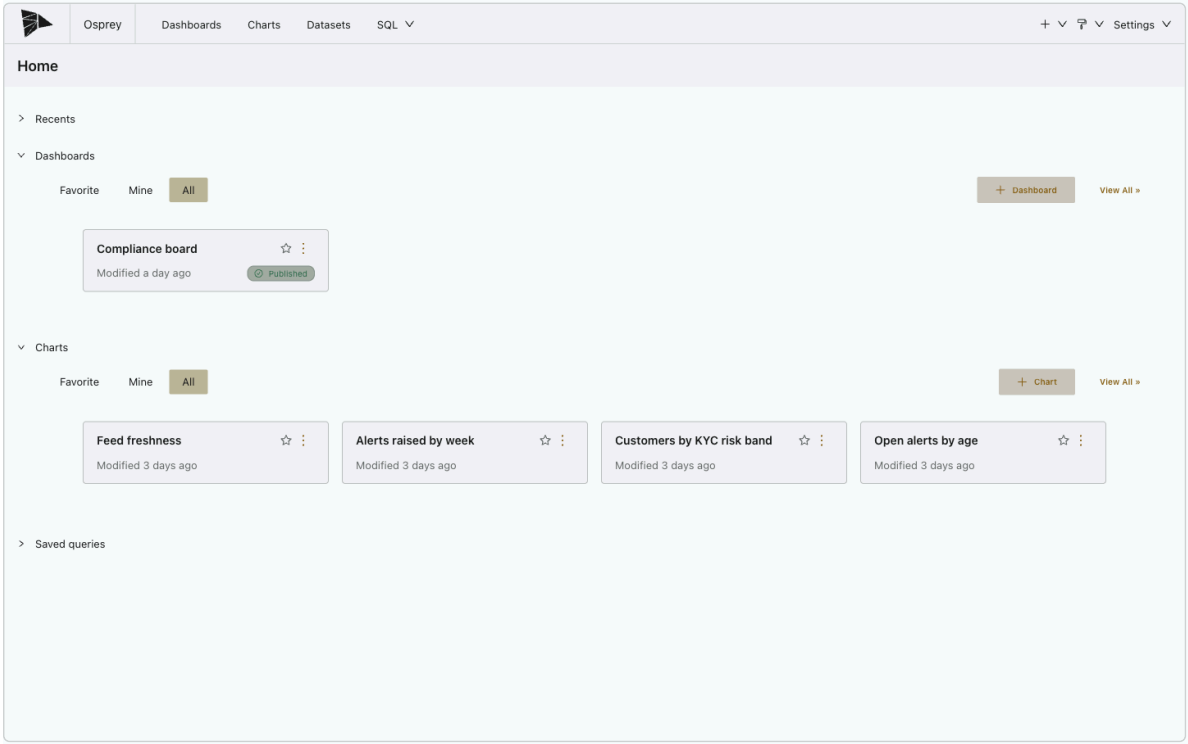


Figure 5. Superset at bi.: the boards and charts built on the reporting figures

13. Backup and restore

Lite. `ospreyflow backup` writes a set to `backups/<time>-<profile>-<commit>/`: the graph store, every PostgreSQL database (the identity provider's accounts as their own export), the analytics database, the object store (each object with its checksum), the configuration, and a manifest of checksums. Taken while the Conduit runs, the stores may be captured at slightly different instants — it says so; `--require-quiesced` refuses to run unless the Conduit is stopped.

`ospreyflow restore <set>` is a **dry run** first: it checks every checksum and estimates how long it will take. `ospreyflow restore <set> --yes` restores. It refuses a set whose checksums do not match, a deployment that is not empty, and a set from another ontology version. Passwords are not restored: the deployment's own are kept.

Small. The cloud protects the data: the database's own backups and snapshots, and the server's daily backup. `ospreyflow backup` refuses a deployment on a managed database, saying so; restore from the cloud's console.

Enterprise. The cloud protects its managed services as for Small. **The graph store and the analytics database run in the cluster, and nothing backs them up yet**: `ospreyflow backup` drives a Compose deployment and does not reach a cluster. Until it does, take snapshots of their volumes with the cloud's own tools.

Audit. The audit log is in the same PostgreSQL as everything else, so it is in the same backup and shares its retention.

14. Updates

Releases are tagged `vMAJOR.MINOR.PATCH`, with signed images and a signed list of every image by digest. Each release's notes are its section of `CHANGELOG.md`.

Which installs `update` moves, and how the others are updated:

Your install	How it is updated
Lite, installed from a release tag	`ospreyflow update` (below)
Lite, built from the checkout (`--build`, or a checkout on no release tag)	`git pull`, then `ospreyflow start --build`. `update` refuses this install, saying so
Small	**Not yet by `update`**: its first step, the backup, refuses a deployment on a managed database (section 13). The cloud's own backups protect the data
Enterprise	Move the checkout to the release's tag and run the install again: the chart is upgraded in place, every password and account kept

Checking for a newer release. The deployment never calls out to look. Run `ospreyflow update --check` on the computer that holds the checkout: it asks the repository's releases and leaves the answer where Admin → System shows it, with when it asked: "*vX.Y.Z* is available", "Up to date", or "The last update check could not answer" and why; before the first check it says "Not checked for updates yet", and an install built from its checkout is told it moves with `git pull` and `ospreyflow start --build`. It checks only when you run it — to be told each week, add it to that computer's own scheduler, for example a cron line `0 8 * * 1 ospreyflow update --check`. If the repository is private, set `GITHUB_TOKEN` to a token that can read it first; without one the check cannot answer, and says so.

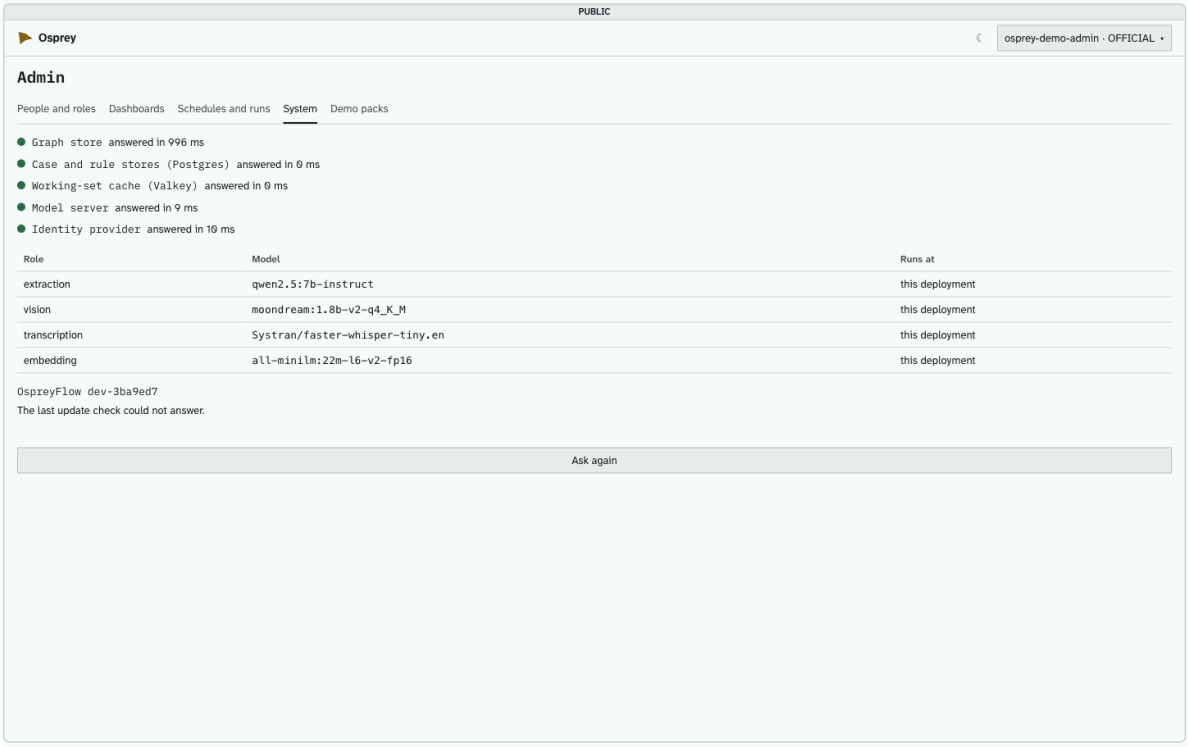


Figure 6. Admin → System: each component's health, the models in use, the version, and the last update check

Moving to a release.

- `ospreyflow update` moves to the newest release (`--to vX.Y.Z` for another; `--yes` asks nothing). It checks every image's signature first and changes nothing if one fails; refuses if a local change to a file would be overwritten ("commit or undo them first"); **backs everything up**; moves the checkout to the tag; starts the release's images and checks each service runs them. If anything fails after the backup it **rolls back** — the previous release and its data restored from the backup just taken — and says so.
- **Without internet**: on a machine with it, `ospreyflow update --make-bundle <file> --to vX.Y.Z` writes the release, verified, to one file; carry it over and run `ospreyflow update --from <file>`, which checks it again with no network.

An update moves forward only.

15. Scheduled and watched pipelines

A published pipeline can run on a **schedule** (hourly, daily, on weekdays or weekly at a time, in a named time zone, or a cron expression; never more often than every 15 minutes) and when a **file arrives** in its dataset's folder (a filename pattern such as `*.csv`; each new file runs once, a minute after it stops changing). Both are set on the Ingest Studio's Publish screen by someone holding `schedule_pipelines`, and run **as that person**, with every check their own run would get. A run that is refused or fails tells its owner in their alerts; three refusals in a row pause it. Admin → Schedules and runs lists every one.

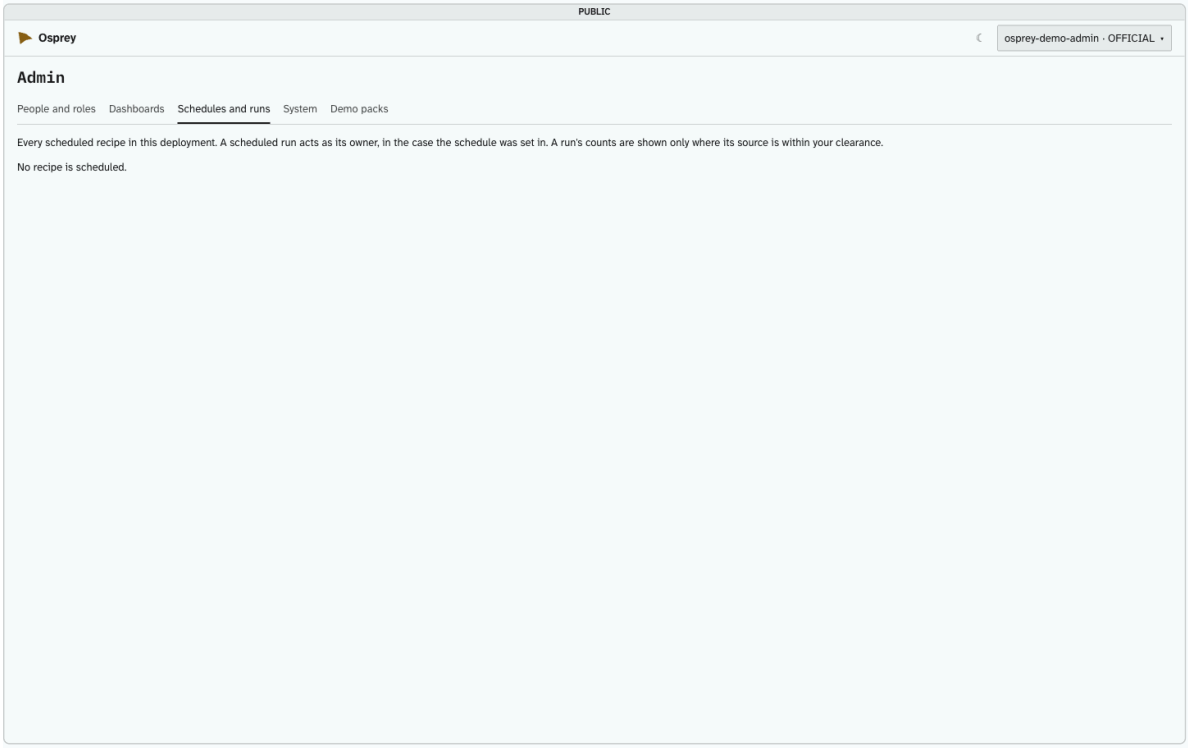


Figure 7. Admin → Schedules and runs

16. When something is wrong

Symptom	Look at
`install` stops at the checks	The crossed line says what to fix; nothing was changed
A service is not healthy	`ospreyflow status`, then `ospreyflow logs <service>`
The page does not open	`ospreyflow status`; that port 80 (Lite) or the DNS records (Small) are right
Nobody can sign in	`ospreyflow logs keycloak`; that the address in `osprey.yaml` is the one in the browser; then `ospreyflow repair-sign-in`
A Small server never answered	Its log, `/var/log/ospreyflow-first-boot.log` (Session Manager on AWS)
An update rolled back	Its own words say why, and where its backup is
The model download stopped with `rename …-partial …: no such file or directory`	A model server restarted mid-download and removed the half-fetched file (fixed from the release after 15 October 2026). Run `ospreyflow start` while the deployment is up: it starts the download again and the server keeps running, so it completes

Appendix A — every `osprey.yaml` setting

Made from the deployment's own schema. Settings marked *section* hold the settings below them.

Option	Value	Default	What it does
`profile`	`local-lite` | `small` | `cluster`	`local-lite`	Which kind of deployment: one machine (local-lite), a cloud server on managed services (small), or a cluster.
`components`	section	—	Which stores the deployment runs, and where their data lives.
`components.graph_store`	`arcadedb` | `janusgraph`	`arcadedb`	The graph store.
`components.analytics_engine`	`duckdb` | `clickhouse`	`duckdb`	Where events are counted: DuckDB inside the Conduit, or ClickHouse shared by several.
`components.search`	`opensearch` | `postgres-fulltext`	`opensearch`	Search; postgres-fulltext is the lighter variant.
`components.storage`	section	—	Folders: inside the containers, and on the host for ingest and tiles.
`components.storage.data`	str	`/var/lib/osprey/data`	Where the deployment keeps its data inside the containers.
`components.storage.staging`	str	`/var/lib/osprey/staging`	Where loads stage their rows before they reach the graph.
`components.storage.backups`	str	`/var/lib/osprey/backups`	Where backups are written.
`components.storage.ingest`	str	`/var/lib/osprey/ingest`	The host folder of source files the Ingest Studio lists (read-only to it; uploads go to its uploaded/ folder).
`components.storage.tiles`	str	`/var/lib/osprey/tiles`	The host folder a built basemap lives in.
`components.source_path`	str or none	none	The owned services (workspace, api, ingest) are PULLED, by the digest recorded in versions.lock, from the registry reference harness/render.py's own service_images names. This is the operator path: the harness image itself carries no services/ or apps/workspace source, and round 05's own finding was that the documented one-command bootstrap could not Launch without it — a clean laptop has no checkout. Set to a checkout root containing services/api/Dockerfile, services/ingest/Dockerfile and apps/workspace/Dockerfile (this repository's own root, for a developer/CI run) to BUILD owned images from source instead. Declared and the Dockerfiles absent is refused by name (harness.render.MissingDockerfileError) — never silently treated as the pull path.
`models`	section	—	The local models, and a client's own model provider if one is chosen.
`models.local_only`	bool	true	No hosted fallback
`models.hosted_fallback`	str or none	none	A provider name, never a credential.
`models.sendable_handling_codes`	list of str	—	Which handling codes may leave for a hosted model.

Option	Value	Default	What it does
`models.provider`	section or none	none	A client's own model provider, reached through the deployment's gateway.
`models.provider.name`	str	required	LiteLLM's name for the provider: openai, anthropic, gemini, azure, bedrock…
`models.provider.roles`	mapping	required	The roles it takes, each with the provider's model for it
`models.provider.api_base`	str or none	none	The provider's address, where it is not LiteLLM's default (an Azure deployment, a proxy)
`models.extraction`	`qwen2.5:7b-instruct` | `qwen2.5:14b-instruct`	`qwen2.5:7b-instruct`	The local model that extracts entities from text. Default: qwen2.5:7b-instruct, the model the product ships with.
`models.vision`	`moondream:1.8b-v2-q4_K_M`	`moondream:1.8b-v2-q4_K_M`	The local model that describes an image and reads its text.
`models.transcription`	`Systran/faster-whisper-tiny.en`	`Systran/faster-whisper-tiny.en`	The local model that transcribes a video's speech.
`models.embedding`	`all-minilm:22m-l6-v2-fp16`	`all-minilm:22m-l6-v2-fp16`	The local model that turns text into vectors.
`governance`	section	—	Identity, handling, ontology and the marking shown on screen.
`governance.identity_provider`	str	`keycloak`	The identity provider people sign in through.
`governance.handling_scheme`	str	`uk-law-enforcement`	The handling scheme every item is marked under.
`governance.ontology`	str	`default`	The ontology the deployment starts from.
`governance.organisation_sign_in`	str or none	none	The alias of an organisation's own identity provider brokered by Keycloak. When set, the welcome page offers *Continue with organisation sign-in*, which goes straight to it. Default: none — one *Sign in* to the OspreyFlow form.
`governance.marking`	str or none	none	The protective marking shown where no case data is on screen: the welcome page, Home, the Ingest Studio and the Admin area; inside a case, the strip shows this or the highest marking on screen, whichever is higher. One of the handling scheme's classifications. Default: the scheme's lowest.
`governance.real_data_exceptions`	str	`config/real_data_exceptions.yaml`	A path to the gitignored list, never its contents.
`cluster`	section	—	A Kubernetes deployment: its namespace, storage classes and machines.
`cluster.hosts`	list of section	—	One machine.
`cluster.hosts.name`	str	required	The machine's name
`cluster.hosts.address`	str	required	Where the machine is reached
`cluster.hosts.roles`	list of str	—	What runs on it
`cluster.namespace`	str	`ospreyflow`	The Kubernetes namespace the deployment is installed in
`cluster.storage_class`	str or none	none	The storage class each store's own volumes use; the cluster's default when unset

Option	Value	Default	What it does
`cluster.shared_storage_class`	str or none	none	Required for a cluster: the storage class of the one volume every Conduit shares (ReadWriteMany) — EFS on AWS, Azure Files on Azure
`network`	section	—	The address the deployment answers at, and what it publishes.
`network.expose_conduit`	bool	false	Publish the Conduit's own port on the host in addition to the workspace's. Not needed to use the deployment — the workspace proxies /v1 — and turning it on widens the exposed surface by a whole tier. For an operator debugging the Conduit directly, or a second client that is not the Glass.
`network.public_origin`	str	`http://ospreyflow.localhost`	The origin a browser uses to reach the Glass; the identity provider is `id.` and the dashboards `bi.` in front of the same host, with the same scheme and port, and the front proxy publishes that port. `06_DECISIONS.md` ADR-050 objection 2's resolution: the identity-provider realm is rendered from this value at apply time and the Glass's public client is registered with an exact redirect URI and post-logout URI derived from it — no wildcards. Change this to move the deployment to another name or port; nothing else needs to agree with it.
`map`	section or none	none	The basemaps the map offers.
`map.default`	str	required	The provider the map opens on, by its id
`map.providers`	list of section	required	One basemap.
`map.providers.id`	str	required	The provider's own name, as map.default names it
`map.providers.label`	str	required	What the Glass's map menu calls it
`map.providers.kind`	`vector` | `raster` | `image` | `dem`	required	
`map.providers.licence`	section	required	The basemap's licence.
`map.providers.licence.id`	str	required	The licence's identifier, e.g. ODbL-1.0 or OGL-UK-3.0
`map.providers.licence.redistributable`	bool	required	Whether the tiles may be passed on to others
`map.providers.licence.client_scope`	`open` | `per-deployment` | `per-seat`	required	Who the licence covers: anyone, this deployment, or each seat
`map.providers.licence.note`	str or none	none	Anything the licence asks to be said
`map.providers.source`	str or none	none	Where the tiles are: a file (file://... pmtiles) or a provider's address; one of source, service and endpoint, or profile
`map.providers.service`	`wmts` | `wms` | `wfs` | `ogc-features` or none	none	
`map.providers.endpoint`	str or none	none	An OGC service's base address (with service), never a tile URL
`map.providers.layer`	str or none	none	The service's layer to show
`map.providers.tile_matrix_set`	str or none	none	WMTS: the tile matrix set

Option	Value	Default	What it does
`map.providers.format`	str or none	none	WMTS: the tile image format
`map.providers.wmts_style`	str or none	none	WMTS: the service's STYLE
`map.providers.zoom_min`	int or none	none	The lowest zoom the provider serves
`map.providers.zoom_max`	int or none	none	The highest zoom the provider serves
`map.providers.profile`	str or none	none	A ready-made provider (providers/<profile>.yaml) to use
`map.providers.variables`	mapping	—	Values the profile's addresses are filled with
`map.providers.credential`	str or none	none	The provider's key, as env:VARIABLE — a name, never the key itself
`map.providers.proxy`	`required` | `recommended` | `none`	`none`	Whether tiles are fetched through the Conduit (which caches them and keeps the key on the server) rather than by the browser
`map.providers.cache`	section or none	none	Keeping fetched tiles a while (online providers only).
`map.providers.cache.ttl_days`	int	required	How long a fetched tile is kept, in days
`map.providers.cache.prefetch_areas`	list of str	—	Areas whose tiles are fetched ahead of use
`map.providers.style`	str or none	none	The map style: bundled:osprey-dark, bundled:osprey-light, or the profile's own
`map.providers.attribution`	str	required	The credit shown on the map, as the licence asks
`reporting`	section	—	Dashboards for the business: report sync and Superset.
`reporting.sync`	bool	false	Run report-sync
`reporting.sync_interval_seconds`	int	`900`	How often each case is pulled into the mart, in seconds.
`reporting.sync_memory`	str	`256m`	Report sync's memory limit.
`reporting.superset`	bool	false	Run Superset
`reporting.superset_memory`	str	`2g`	Superset's memory limit.
`mail`	section or none	none	A mail server, for invitations and password resets by email.
`mail.host`	str	required	The mail server's name
`mail.port`	int	`587`	The mail server's port.
`mail.from_address`	str	required	The address invitations come from
`mail.starttls`	bool	true	Whether mail is sent over TLS (STARTTLS).
`mail.username`	str or none	none	A relay that takes mail without signing in. With one, the password comes from OSPREY_SMTP_PASSWORD at install.
`welcome`	section	—	The welcome page before sign-in: its About section and footer links.
`welcome.about`	bool	true	Off for an internal-only deployment.
`welcome.account_help`	str or none	none	Where people get help with their account.
`welcome.accessibility`	str or none	none	The organisation's accessibility statement.
`welcome.privacy`	str or none	none	The organisation's privacy notice.

Option	Value	Default	What it does
`welcome.service_status`	str or none	none	A page saying whether the service is up.
`external`	section	—	What a cloud runs for the deployment instead of a container (Small).
`external.postgres`	section or none	none	A cloud's managed PostgreSQL; its password arrives at install.
`external.postgres.host`	str	required	The managed server's address
`external.postgres.port`	int	`5432`	Its port
`external.postgres.database`	str	`ospreyflow`	The deployment's database on it
`external.postgres.user`	str	`osprey`	The login, which needs CREATEROLE and CREATEDB there
`external.postgres.sslmode`	`require` | `verify-ca` | `verify-full`	`require`	A managed server is reached over TLS.
`external.object_store`	section or none	none	A cloud's S3, for every S3-protocol store (AWS).
`external.object_store.endpoint`	str	required	Host and port, no scheme: always HTTPS
`external.object_store.region`	str	required	The region the buckets live in
`external.object_store.credentials`	`instance` | `keys`	`instance`	The server's own cloud identity, no key anywhere; `keys` reads OSPREY_S3_ACCESS_KEY and OSPREY_S3_SECRET_KEY at apply.
`external.object_store.bucket_prefix`	str	`ospreyflow-`	Each bucket is <prefix><name>, as OpenTofu made them
`external.media`	section or none	none	Media on Azure Blob; its key arrives at install.
`external.media.blob_account`	str	required	The Blob storage account
`external.media.blob_container`	str	`media`	The container media is kept in
`external.valkey`	section or none	none	A cloud's managed Valkey or Redis (cluster only).
`external.valkey.host`	str	required	The managed cache's address
`external.valkey.port`	int	`6379`	Its port
`external.valkey.tls`	bool	true	Whether it is reached over TLS
`external.search`	section or none	none	A cloud's managed OpenSearch (cluster on AWS only).
`external.search.url`	str	required	The domain's HTTPS address